

## MLDL Experiment 11

**Aim:** Classification of Mental Workload Using Brain Connectivity & Machine Learning on EEG Data.

Slides:

# Classification of Mental Workload Using Brain Connectivity & Machine Learning on EEG Data

Mansi Davkhar

Guide: Mrs. Kajal Joseph

Vivekanand Education Society's Institute of Technology

INFT / D15C / 24

## Introduction



### What is Mental Workload?

The cognitive effort required to perform tasks — influenced by complexity, time pressure, and environment.



### Why Does It Matter?

Excessive workload leads to stress, fatigue, errors, and accidents in critical domains like aviation, medicine, and driving.



### Why EEG?

Non-invasive, high temporal resolution, real-time assessment, no subject bias — ideal for continuous cognitive monitoring.



### Our Approach

Combine brain effective connectivity (dDTF) with SVM to automatically classify Low vs Medium vs High MWL states.

# Problem Statement

## The Core Challenge

*How can we automatically and objectively measure a person's mental workload level in real-time using brain signals?*

### Specifically:

Classify 3 levels of MWL (Low / Medium / High) from 14-channel EEG data collected during a multitasking test, using brain connectivity patterns as features.

## Real-World Impact

- ✈️ Aviation — pilot overload detection
- 🏥 Medicine — surgeon fatigue alerts
- 🚗 Autonomous vehicles — driver monitoring
- 🎓 Education — adaptive learning systems
- 🏭 Industry — operator safety systems

48

EEG subjects

14

Brain channels

3

MWL classes

83.25%

Our accuracy

0.94

AUC score

3

# Objectives & Expected Outcomes

01

## Preprocess EEG Signals

Apply highpass filter, notch filter, ASR artifact removal, and average re-referencing to clean raw 14-channel EEG data.

02

## Extract Brain Connectivity Features

Compute effective connectivity using dDTF (direct Directed Transfer Function) across 5 frequency bands from MVAR model.

03

## Hierarchical Feature Selection

Select top-30 per band (AUC-based) → 150 features → Forward Feature Selection to identify optimal 25-feature subset.

04

## Train & Evaluate SVM Classifier

Train RBF-kernel SVM on 3-class MWL problem. Evaluate with 7-fold stratified cross-validation. Compare with paper benchmarks.

05

## Interpret Results

Identify which brain regions and frequency bands are most discriminative for mental workload classification.

4

# Literature Review

| Reference             | Year | Feature Method            | Classifier | Accuracy |
|-----------------------|------|---------------------------|------------|----------|
| Lim et al. [STEW]     | 2018 | Power Spectral Density    | SVR        | 79.5%    |
| Pandey et al.         | 2020 | Not specified             | RF         | 58.52%   |
| Pandey et al.         | 2020 | Not specified             | KNN        | 61.08%   |
| Chakladar et al.      | 2020 | Freq + Linear + NonLinear | RF         | 83.0%    |
| Chakladar et al.      | 2020 | Freq + Linear + NonLinear | SVM        | 83.33%   |
| Zhu et al.            | 2021 | Graph features (EEG)      | SVM        | 89.6%    |
| Safari et al. (paper) | 2024 | dDTF Connectivity         | SVM        | 89.53%   |
| Our Implementation    | 2026 | dDTF + 3-class SVM        | SVM        | 83.25%   |

Research Gap: Existing approaches use 2-class classification. We extend to 3-class (Low/Med/High) with multiband dDTF connectivity features — a more realistic and challenging problem.

7

## DATASET

# Dataset — STEW (STEW EEG Workload)

### STEW Dataset Details

|                |                                                |
|----------------|------------------------------------------------|
| Source:        | IEEE DataPort (open-access)                    |
| Subjects:      | 48 healthy male subjects                       |
| Device:        | Emotiv EPOC headset                            |
| Channels:      | 14 EEG + 2 reference (CMS/DRL)                 |
| Sampling Rate: | 128 Hz, 16-bit A/D                             |
| Task:          | SIMKAP multitasking test                       |
| Conditions:    | Low MWL: 2.5 min rest   High MWL: 2.5 min task |
| Labels:        | class_012.mat (0=Low, 1=Med, 2=High)           |
| File Size:     | ~34 GB uncompressed                            |

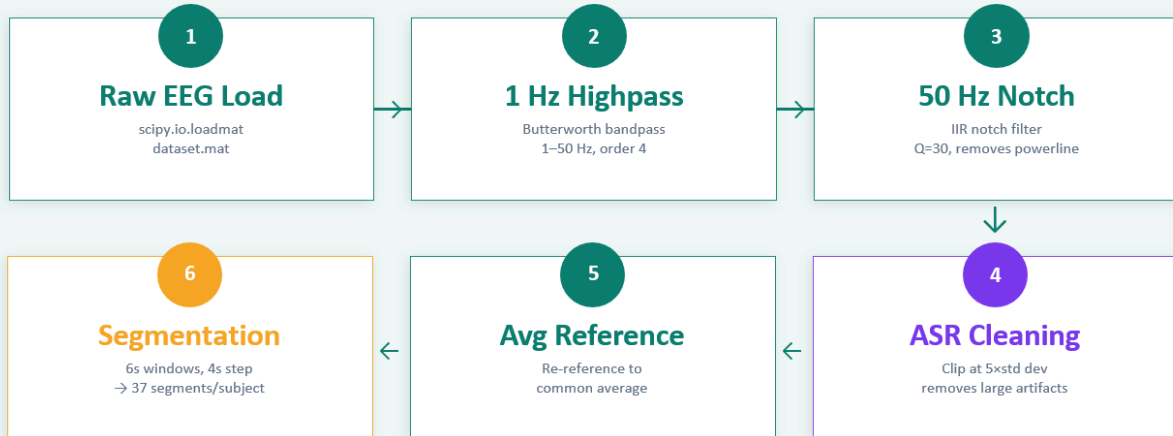
### 14-Channel Placement

|                                                       |
|-------------------------------------------------------|
| <b>Frontal-prefrontal</b><br>AF3, AF4, F7, F3, F4, F8 |
| <b>Frontocentral</b><br>FC5, FC6                      |
| <b>Temporal</b><br>T7, T8                             |
| <b>Parietal</b><br>P7, P8                             |
| <b>Occipital</b><br>O1, O2                            |

10–20 international electrode placement system

8

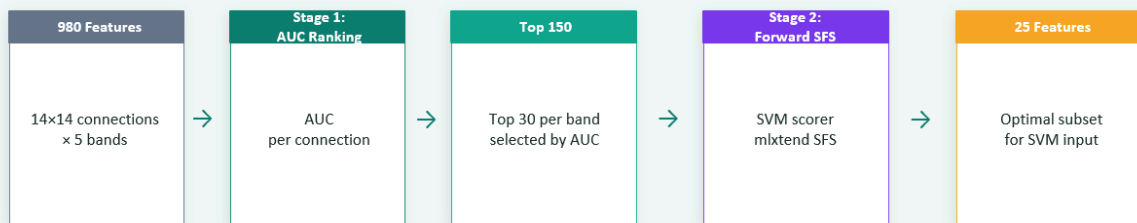
## Data Preprocessing Pipeline



Output: Clean EEG segments, shape (14 channels × 768 samples) per 6-second window.  
Total: 48 subjects × 2 conditions × 37 windows = 3,552 EEG segments processed.

9

## Hierarchical Feature Selection



Top Connections by Band (from our analysis, AUC values)

| Band             | Top Connection 1 | Top Connection 2 | Top Connection 3 |
|------------------|------------------|------------------|------------------|
| <b>Delta (δ)</b> | F7→F8 (0.756)    | FC5→F8 (0.717)   | O2→F8 (0.713)    |
| <b>Theta (θ)</b> | F7→F8 (0.728)    | O2→F8 (0.716)    | AF3→F8 (0.697)   |
| <b>Alpha (α)</b> | FC5→O2 (0.675)   | FC6→O1 (0.663)   | FC6→O2 (0.663)   |
| <b>Beta (β)</b>  | P8→F8 (0.670)    | AF4→F8 (0.666)   | O2→F8 (0.666)    |
| <b>Gamma (γ)</b> | O2→F8 (0.688)    | P8→F8 (0.670)    | O2→FC6 (0.669)   |

Key finding: Frontal lobe (F8, FC6) consistently dominates as the target of information flow across all bands — consistent with prior MWL research.

11

## SVM Model — Architecture & Configuration

### SVM Configuration

|                                         |                                     |
|-----------------------------------------|-------------------------------------|
| <b>Kernel:</b>                          | RBF (Radial Basis Function)         |
| <b>Classes:</b>                         | 3 (Low / Medium / High MWL)         |
| <b>Cross-validation:</b>                | 7-fold Stratified CV                |
| <b>Scaler:</b>                          | StandardScaler (z-score)            |
| <b>Probability:</b>                     | Enabled (predict_proba)             |
| <b>Grid Search C:</b>                   | [0.1, 1, 10, 100]                   |
| <b>Grid Search <math>\gamma</math>:</b> | ['scale', 'auto', 0.001, 0.01, 0.1] |

### Why SVM with RBF kernel?

- Maps features to higher-dim space via kernel trick — handles non-linear boundaries between MWL classes
- Maximises margin between classes — more robust to noise in EEG-derived features
- Effective on small datasets (1665 samples, 25 features)
- Probabilistic output enables multi-class ROC-AUC calculation
- Grid search found  $C=1$ ,  $\gamma=0.1$  optimal — balanced regularisation

Class distribution — Low MWL: 481 samples | Medium MWL: 666 samples | High MWL: 518 samples | Total: 1,665 samples

12

## System Architecture

### Raw EEG (dataset.mat)

48 subjects  $\times$  14 ch  $\times$  19200 samples

### Preprocessing

Highpass · Notch · ASR · AvgRef

### Windowing

6s window, 4s step  $\rightarrow$  37 segs/subject

### MVAR Fitting

AIC order selection, statsmodels VAR

### dDTF Extraction

$H(f)$  via FFT, 5 frequency bands

### AUC Feature Ranking

Top 30/band  $\rightarrow$  150 features (LDA AUC)

### Forward FS

SVM scorer, 7-fold CV  $\rightarrow$  25 features

### SVM Classifier

RBF kernel,  $C=1$ ,  $\gamma=0.1$

### 3-Class Prediction

Low / Medium / High MWL

### Key Numbers

**3,552**

EEG windows processed

**980**

Raw features per window

**150**

After AUC stage-1 selection

**25**

Final features for SVM

**1,665**

Total labelled samples

**7-fold**

Cross-validation strategy

Python · scikit-learn · statsmodels · mlxtend · scipy

13

## Tools & Technologies



### Python 3.10+

Core language

Primary development environment



### scikit-learn

ML framework

SVM, GridSearchCV, metrics, cross-validation



### statsmodels

Timeseries

VAR model fitting, AIC order selection



### mlxtend

Feature selection

SequentialFeatureSelector (SFS)



### NumPy / SciPy

Numerics

Array ops, signal filtering, FFT, .mat loading



### Matplotlib / Seaborn

Visualisation

Confusion matrix, metrics bar plots, results



### joblib

Persistence

Model serialisation (svm\_model\_final.pkl)



### Jupyter Notebook

Dev environment

Interactive cell-by-cell execution with caching

14

## Training Process & Hyperparameters

### 7-Fold Stratified Cross-Validation

|       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|
| TEST  | train | train | train | train | train | train |
| train | TEST  | train | train | train | train | train |
| train | train | TEST  | train | train | train | train |
| train | train | train | TEST  | train | train | train |
| train | train | train | train | TEST  | train | train |
| train | train | train | train | train | TEST  | train |
| train | train | train | train | train | train | TEST  |

Orange = Test fold Teal = Training folds Each fold: ~238 samples test, ~1427 train

|        |                 |        |
|--------|-----------------|--------|
| Fold 1 | SVM C:          | 1      |
| Fold 2 | SVM γ:          | 0.1    |
| Fold 3 | Kernel:         | RBF    |
| Fold 4 | CV folds:       | 7      |
| Fold 5 | Grid C vals:    | 4      |
| Fold 6 | Grid γ vals:    | 5      |
| Fold 7 | Total fits:     | 100    |
|        | Best inner acc: | 82.76% |

### Per-Fold Results

|            |            |            |            |            |            |            |
|------------|------------|------------|------------|------------|------------|------------|
| Fold 1     | Fold 2     | Fold 3     | Fold 4     | Fold 5     | Fold 6     | Fold 7     |
| 83.19%     | 83.19%     | 79.41%     | 82.35%     | 80.67%     | 86.13%     | 87.76%     |
| AUC:0.9347 | AUC:0.9387 | AUC:0.9168 | AUC:0.9550 | AUC:0.9250 | AUC:0.9428 | AUC:0.9580 |

15

# Results

| Mean Accuracy | Mean Precision | Mean Recall | Mean F1 Score |
|---------------|----------------|-------------|---------------|
| 83.25%        | 83.88%         | 82.88%      | 83.15%        |
| ± 2.70%       | ± 3.00%        | ± 2.69%     | ± 2.74%       |

|                            |        |                                                                         |
|----------------------------|--------|-------------------------------------------------------------------------|
| Mean ROC-AUC (OVR, macro): | 0.9399 | Excellent discriminability — model strongly separates the 3 MWL classes |
|----------------------------|--------|-------------------------------------------------------------------------|

Classification Report (7-fold CV aggregated predictions)

| Class          | Precision | Recall | F1-Score | Support |
|----------------|-----------|--------|----------|---------|
| Low MWL (0)    | 0.8519    | 0.8206 | 0.8360   | 481     |
| Medium MWL (1) | 0.8258    | 0.8859 | 0.8548   | 666     |
| High MWL (2)   | 0.8411    | 0.8069 | 0.8237   | 518     |
| Macro avg      | 0.8396    | 0.8378 | 0.8382   | 1665    |

16

## EVALUATION

### Performance Evaluation — Confusion Matrix

|        |          | Predicted → |         |          | Class-wise accuracy |
|--------|----------|-------------|---------|----------|---------------------|
|        |          | Low (0)     | Med (1) | High (2) |                     |
| ↓ True | Low (0)  | 391         | 57      | 33       | 81.3%               |
|        | Med (1)  | 34          | 577     | 55       | 86.6%               |
|        | High (2) | 30          | 70      | 418      | 80.7%               |

#### Key Insights

- Medium MWL has most samples (666) and best recall (88.6%)
- Low MWL: some confusion with Medium (57 misclassified)
- High MWL: strong detection despite smaller class
- Diagonal dominant — model reliably separates all 3 classes
- No catastrophic misclassification (Low↔High very rare)

|                         |                          |                       |                   |                 |
|-------------------------|--------------------------|-----------------------|-------------------|-----------------|
| Accuracy: 83.25% ± 2.70 | Precision: 83.88% ± 3.00 | Recall: 82.88% ± 2.69 | F1: 83.15% ± 2.74 | ROC-AUC: 0.9399 |
|-------------------------|--------------------------|-----------------------|-------------------|-----------------|

17

## Advantages & Limitations

### ✓ Advantages

- Effective connectivity captures directional brain signal flow — richer than simple correlation or PSD features
- Hierarchical feature selection (AUC → SFS) dramatically reduces dimensionality from 980 → 25 features
- ROC-AUC of 0.94 shows excellent class discrimination despite only 83% accuracy
- Disk caching strategy allows safe resumption — critical for long MVAR computation on 16 GB RAM
- Extends paper's binary classification to realistic 3-class problem
- SVM with RBF generalises well on small EEG datasets without overfitting

### ⚠ Limitations

- STEW dataset: only 48 male subjects — limited gender diversity affects generalisability
- MVAR computation is extremely time-intensive (2–4 hours CPU) — not suitable for real-time deployment
- 3-class accuracy (83.25%) is lower than paper's binary result (89.53%) — harder problem
- Python MVAR fitting less optimised than MATLAB's SIFT toolbox used in paper
- Small dataset (1665 samples) — risk of variance across folds (std  $\pm 2.7\%$ )
- No deep learning or attention mechanisms explored that could further improve accuracy

18

## Real-World Applications



### Aviation

Real-time pilot cognitive overload detection to trigger autopilot assistance or alerts during high-demand flight phases.



### Clinical Medicine

Monitor surgeon or radiologist fatigue during long procedures. Alert when workload reaches unsafe threshold.



### Automotive

Driver mental state monitoring in autonomous vehicles. Adapt assistance level based on detected cognitive load.



### Adaptive Education

Adjust difficulty and pacing of e-learning content in real time based on student's estimated mental workload.



### Gaming / UX Research

Assess frustration and engagement in game design. Optimise difficulty curves using physiological signals.



### Industrial Safety

Monitor control room operators in nuclear plants, refineries. Enforce mandatory breaks when overload is detected.

19



## Future Scope

|    |                                                                                                                                                   |      |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------|------|
| 01 | <b>Deep Learning Models</b><br>Apply CNN/LSTM or Transformer architectures directly on dDTF connectivity images — the paper itself suggests this. | High |
| 02 | <b>Real-Time Pipeline</b><br>Optimise MVAR fitting with GPU acceleration (cupy) or online/recursive VAR for real-time BCIs.                       | High |
| 03 | <b>Multi-Modal Fusion</b><br>Combine EEG with ECG, fNIRS, or eye-tracking for more robust multimodal MWL assessment.                              | Med  |
| 04 | <b>Diverse Datasets</b><br>Extend to mixed-gender, different age groups, and clinical populations for better generalisation.                      | Med  |
| 05 | <b>Personalised Models</b><br>Subject-specific fine-tuning using transfer learning — EEG signals vary significantly between individuals.          | Med  |
| 06 | <b>Continuous Monitoring</b><br>Deploy as a wearable BCI application with Emotiv EPOC and a Raspberry Pi edge inference node.                     | Low  |

20

## Conclusion



Successfully implemented a 3-class mental workload classification pipeline on the STEW EEG dataset using dDTF brain connectivity as features.



Hierarchical feature selection (AUC ranking → Forward SFS) reduced 980 raw features to 25 optimal features, improving interpretability.



SVM with RBF kernel ( $C=1$ ,  $\gamma=0.1$ ) achieved 83.25% accuracy and ROC-AUC of 0.94 in 7-fold CV — strong discriminability across all 3 classes.



Frontal lobe electrodes (F8, FC6) dominate as targets of information flow — consistent with established neuroscience of executive function.



Future work should explore CNN/Transformer models on dDTF connectivity images and GPU-accelerated real-time inference pipelines.

Accuracy: 83.25% | ROC-AUC: 0.9399 | F1: 83.15%

21

## References

- [1] Safari, M.R., Shalbaf, R., Bagherzadeh, S. & Shalbaf, A. (2024). Classification of mental workload using brain connectivity and machine learning on EEG data. Scientific Reports, 14, 9153. <https://doi.org/10.1038/s41598-024-59652-w>
- [2] Lim, W., Sourina, O. & Wang, L. (2018). STEW: Simultaneous task EEG workload dataset. IEEE Trans. Neural Syst. Rehabil. Eng., 26(10), 2106–2114.
- [3] Korzeniewska, A. et al. (2003). Determination of information flow direction among brain structures by a modified directed transfer function (dDTF) method. J. Neurosci. Methods, 125, 195–207.
- [4] Mullen, T. (2010). Source Information Flow Toolbox (SIFT). Swartz Center for Computational Neuroscience, UCSD.
- [5] Zhu, G., Zong, F., et al. (2021). Cognitive load during multitasking assessed using single-channel EEG and graph methods. IEEE Access, 9, 33102–33109.
- [6] Longo, L. et al. (2022). Human mental workload: A survey and a novel inclusive definition. Front. Psychol., 13, 969140.
- [7] Han, M. & Liu, X. (2012). Forward feature selection based on approximate Markov blanket. ISNN 2012. Lecture Notes in Computer Science, 7368.
- [8] Scikit-learn (2024). Machine Learning in Python. <https://scikit-learn.org>

22

# Thank You!

## Questions & Discussion

**83.25%**

Accuracy

**0.9399**

ROC-AUC

**25**

Features

**3**

MWL Classes

Mansi Davkhar · VESIT · 2026

## Output:

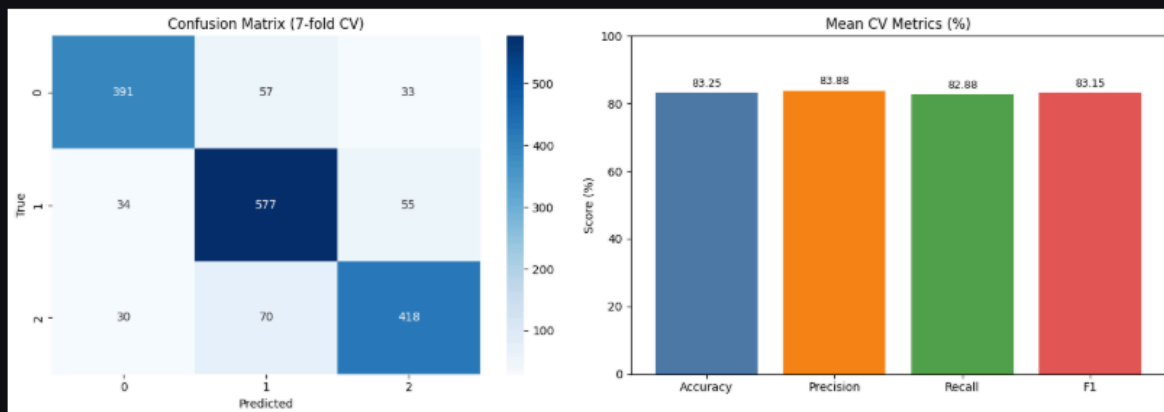
```
Final feature matrix: (1665, 25)
Class distribution: [481 666 518]

Grid searching SVM hyperparameters (C, gamma)...
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best params: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
Best inner CV accuracy: 0.8276
```

```
Running 7-fold CV with best SVM...
Fold 1: Acc=83.19% AUC(OVR)=0.9347
Fold 2: Acc=83.19% AUC(OVR)=0.9387
Fold 3: Acc=79.41% AUC(OVR)=0.9168
Fold 4: Acc=82.35% AUC(OVR)=0.9550
Fold 5: Acc=80.67% AUC(OVR)=0.9250
Fold 6: Acc=86.13% AUC(OVR)=0.9428
Fold 7: Acc=87.76% AUC(OVR)=0.9665
```

```
=====
Mean Accuracy: 83.25% ± 2.70
Mean Precision: 83.88% ± 3.00
Mean Recall: 82.88% ± 2.69
Mean F1: 83.15% ± 2.74
Mean ROC-AUC: 0.9399
```

```
=====
Model saved: svm_model_final.pkl
Metrics saved: final_metrics.npz
```



```

Classification report (overall CV predictions):

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.8593    | 0.8129 | 0.8355   | 481     |
| 1            | 0.8196    | 0.8664 | 0.8423   | 666     |
| 2            | 0.8261    | 0.8069 | 0.8164   | 518     |
| accuracy     |           |        | 0.8324   | 1665    |
| macro avg    | 0.8350    | 0.8287 | 0.8314   | 1665    |
| weighted avg | 0.8331    | 0.8324 | 0.8323   | 1665    |

### Code:

```

import os
from typing import List

import joblib
import numpy as np
import pandas as pd
import streamlit as st
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix, f1_score

try:
    import plotly.express as px
    import plotly.graph_objects as go
    PLOTLY_OK = True
except Exception:
    PLOTLY_OK = False

st.set_page_config(page_title="Mental Workload
Demo", layout="wide")

#
-----
# Config
#
-----
ROOT_DIR =
os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(ROOT_DIR,
"STEW_Dataset")

```

```

CACHE_DIR = os.path.join(DATA_DIR, "cache")

CH_NAMES = ["AF3", "F7", "F3", "FC5", "T7",
"P7", "O1", "O2", "P8", "T8", "FC6", "F4", "F8",
"AF4"]
BAND_NAMES = ["delta", "theta", "alpha", "beta",
"gamma"]
N_CH = 14

# Expected cache files from your notebook pipeline
PATH_X_ALL = os.path.join(CACHE_DIR,
"X_all_features.npz")
PATH_X_TOP150 = os.path.join(CACHE_DIR,
"X_top150.npz")
PATH_X_SELECTED = os.path.join(CACHE_DIR,
"X_selected_sfs.npz")
PATH_METRICS = os.path.join(CACHE_DIR,
"final_metrics.npz")
PATH_MODEL = os.path.join(CACHE_DIR,
"svm_model_final.pkl")

def _check_required_files() -> List[str]:
    missing = []
    for p in [PATH_X_SELECTED, PATH_MODEL]:
        if not os.path.exists(p):
            missing.append(p)
    return missing

@st.cache_data(show_spinner=False)
def load_selected_data():

```

```

data = np.load(PATH_X_SELECTED)
X = data["X"]
y = data["y"].astype(int)
idx = data["indices"].astype(int) if "indices" in
data.files else None
return X, y, idx

@st.cache_resource(show_spinner=False)
def load_model():
    return joblib.load(PATH_MODEL)

@st.cache_data(show_spinner=False)
def load_metrics_if_available():
    if not os.path.exists(PATH_METRICS):
        return None
    # Return plain serializable types for st.cache_data.
    with np.load(PATH_METRICS) as data:
        return {k: data[k] for k in data.files}

def build_all_feature_names():
    names = []
    for band in BAND_NAMES:
        for i, ch_i in enumerate(CH_NAMES):
            for j, ch_j in enumerate(CH_NAMES):
                names.append(f"{ch_j}->{ch_i}_{band}")
    return names

def build_selected_feature_names(selected_indices):
    if selected_indices is None:
        return []
    all_names = build_all_feature_names()
    out = []
    for i in selected_indices:
        if 0 <= int(i) < len(all_names):
            out.append(all_names[int(i)])
        else:
            out.append(f"feature_{int(i)}")
    return out

def infer_window_layout(n_samples,
default_recordings=45):
    # The pipeline usually generates 37
    windows/recording for 45 recordings.
    # Infer safely from data length for robustness.

```

```

    if n_samples % default_recordings == 0:
        return default_recordings, n_samples //
default_recordings
    # Fallback: make a near-square layout for demo
    replay.
    rec = max(1, int(np.sqrt(n_samples)))
    win = max(1, n_samples // rec)
    return rec, win

#
-----
# UI Header
#
-----
st.title("Mental Workload Classification Demo")
st.caption("Offline replay demo using cached features
and trained SVM artifacts")

missing = _check_required_files()
if missing:
    st.error("Missing required cache/model files. Run
notebook training cells first:")
    for m in missing:
        st.write(f"- {m}")
    st.stop()

X_sel, y_sel, selected_indices = load_selected_data()
model = load_model()
metrics_npz = load_metrics_if_available()
selected_feature_names =
build_selected_feature_names(selected_indices)

n_classes = len(np.unique(y_sel))
class_counts = np.bincount(y_sel)

rec_count, windows_per_rec =
infer_window_layout(len(y_sel),
default_recordings=45)

#
-----
# Sidebar
#
-----

```

```

st.sidebar.header("Demo Controls")
st.sidebar.write(f'Samples: {len(y_sel)}')
st.sidebar.write(f'Selected features:
{X_sel.shape[1]}')
st.sidebar.write(f'Classes: {list(np.unique(y_sel))}')

show_probs = st.sidebar.checkbox("Show class
probabilities", value=True)

#
-----
# Tabs
#
-----
tab1, tab2, tab3, tab4 = st.tabs([
    "Overview",
    "Performance",
    "Feature Insights",
    "Replay Prediction",
])

with tab1:
    c1, c2, c3 = st.columns(3)
    c1.metric("Total Windows", f'{len(y_sel)}')
    c2.metric("Selected Features",
f'{X_sel.shape[1]}')
    c3.metric("Classes", f'{n_classes}')

    st.subheader("Class Distribution")
    df_class = pd.DataFrame({
        "class": np.arange(len(class_counts)),
        "count": class_counts,
    })

    if PLOTLY_OK:
        fig = px.bar(df_class, x="class", y="count",
text="count", title="Samples per Class")
        st.plotly_chart(fig, use_container_width=True)
    else:
        st.bar_chart(df_class.set_index("class"))

    st.info(
        "This dashboard is a replay demo over cached
windows. "
        "It demonstrates full pipeline behavior without
live EEG acquisition."
    )

```

```

with tab2:
    st.subheader("Cross-Validated Performance")

    if metrics_npz is not None and all(k in metrics_npz
for k in ["acc", "f1", "y_true", "y_pred"]):
        acc = metrics_npz["acc"]
        f1 = metrics_npz["f1"]
        y_true = metrics_npz["y_true"].astype(int)
        y_pred = metrics_npz["y_pred"].astype(int)
    else:
        # Fallback: quick estimate on full data (not CV),
still useful for demo continuity.
        y_pred = model.predict(X_sel).astype(int)
        y_true = y_sel
        acc = np.array([accuracy_score(y_true, y_pred)
* 100])
        f1 = np.array([f1_score(y_true, y_pred,
average="macro") * 100])

        c1, c2 = st.columns(2)
        c1.metric("Mean Accuracy (%)",
f'{np.mean(acc):.2f}')
        c2.metric("Mean Macro F1 (%)",
f'{np.mean(f1):.2f}')

        cm = confusion_matrix(y_true, y_pred)
        cm_df = pd.DataFrame(cm)

        if PLOTLY_OK:
            fig_cm = px.imshow(
                cm_df,
                text_auto=True,
                color_continuous_scale="Blues",
                title="Confusion Matrix",
                labels={"x": "Predicted", "y": "True", "color":
"Count"},
            )
            st.plotly_chart(fig_cm,
use_container_width=True)
        else:
            st.dataframe(cm_df)

    st.subheader("Classification Report")
    report = classification_report(y_true, y_pred,
output_dict=True)
    report_df = pd.DataFrame(report).T
    st.dataframe(report_df, use_container_width=True)

```

```

with tab3:
    st.subheader("Selected Features")

    if selected_feature_names:
        df_feat = pd.DataFrame({
            "rank": np.arange(1,
len(selected_feature_names) + 1),
            "feature": selected_feature_names,
        })
        st.dataframe(df_feat, use_container_width=True,
height=340)

        # Band frequency among selected features
        bands = [f.split("_")[-1] if "_" in f else
"unknown" for f in selected_feature_names]
        df_band =
pd.Series(bands).value_counts().rename_axis("band")
.reset_index(name="count")

        if PLOTLY_OK:
            fig_band = px.pie(df_band, names="band",
values="count", title="Band Composition of Selected
Features")
            st.plotly_chart(fig_band,
use_container_width=True)
        else:
            st.bar_chart(df_band.set_index("band"))
        else:
            st.warning("Selected feature names are
unavailable (missing indices in
X_selected_sfs.npz).")

with tab4:
    st.subheader("Replay Window Prediction")

    st.write(
        "Pick a recording and window to simulate
real-time inference over previously processed
windows."
    )

    rec_id = st.slider("Recording ID", min_value=0,
max_value=max(rec_count - 1, 0), value=0)
    win_id = st.slider("Window ID", min_value=0,
max_value=max(windows_per_rec - 1, 0), value=0)

    global_idx = rec_id * windows_per_rec + win_id
    if global_idx >= len(y_sel):

```

```

        st.warning("Selected replay index is outside
available range. Adjust sliders.")
    else:
        x = X_sel[global_idx].reshape(1, -1)
        pred = int(model.predict(x)[0])

        c1, c2 = st.columns(2)
        c1.metric("Predicted Class", f"{pred}")
        c2.metric("True Class",
f"{int(y_sel[global_idx])}")

        if show_probs and hasattr(model,
"predict_proba"):
            probs = model.predict_proba(x)[0]
            df_prob = pd.DataFrame({"class":
np.arange(len(probs)), "probability": probs})

            if PLOTLY_OK:
                fig_prob = px.bar(
                    df_prob,
                    x="class",
                    y="probability",
                    text="probability",
                    title=f"Class Probabilities (sample index
{global_idx})",
                    range_y=[0, 1],
                )

            fig_prob.update_traces(texttemplate="%{text:.3f}",
textposition="outside")
            st.plotly_chart(fig_prob,
use_container_width=True)
        else:
            st.bar_chart(df_prob.set_index("class"))

        # Timeline preview for selected recording
        start = rec_id * windows_per_rec
        end = min(start + windows_per_rec, len(y_sel))
        rec_true = y_sel[start:end]
        rec_pred =
model.predict(X_sel[start:end]).astype(int)

        df_tl = pd.DataFrame({
            "window": np.arange(len(rec_true)),
            "true": rec_true,
            "pred": rec_pred,
        })

```

```

st.markdown("**Window-wise trend for
selected recording**")
if PLOTLY_OK:
    fig_tl = go.Figure()

fig_tl.add_trace(go.Scatter(x=df_tl["window"],
y=df_tl["true"], mode="lines+markers",
name="True"))

fig_tl.add_trace(go.Scatter(x=df_tl["window"],
y=df_tl["pred"], mode="lines+markers",
name="Pred"))
    fig_tl.update_layout(yaxis_title="Class",
xaxis_title="Window", height=340)
    st.plotly_chart(fig_tl,
use_container_width=True)
else:
    st.line_chart(df_tl.set_index("window"))

st.success("Demo app loaded successfully.")

```